Documentação da solução e implementação Java

EasifyAI

Análise de conversas comerciais B2B com inteligência artificial, regras de negócio e persistência Oracle.

Disciplina: Domain Driven Design — Java
Instituição: FIAP
Equipe: EasifyAI
Entrega: Projeto Java finalizado

Clara Barboza Costa — RM 561490
Lucas Santana Silva — RM 566261
Pedro Henrique Lamin Rodrigues — RM 566379
Tomas Antonio Braga Cantuaria — RM 563295

São Paulo • 2026
Versão verificada em 07 de outubro de 2026

Sumário e objetivo

Seção	Página
Objetivo e escopo	2
Arquitetura e responsabilidades	3
Modelagem UML do domínio	4
Mapeamento das cinco tabelas Oracle	5
Contrato REST e exemplos	6
Validações e regras de negócio	7
Execução, testes e atendimento dos requisitos	8

Objetivo

Reduzir o esforço de leitura de transcrições comerciais e padronizar informações úteis para vendas B2B: dores, interesse, risco de churn, concorrentes, produto recomendado e próximas ações.

Escopo implementado

O sistema recebe uma transcrição, consulta a Gemini, aplica regras determinísticas do domínio e salva a análise no Oracle. A API permite consultar, corrigir e excluir análises, recuperar o texto original e acrescentar eventos ao histórico de acompanhamento.

A entrega contém somente o backend. A coleção Postman permite demonstrar o fluxo completo sem front-end. Não foram incluídos autenticação, paginação, dashboards ou novos processos comerciais.

Arquitetura e responsabilidades

Camada	Responsabilidade e classes principais
Entrada HTTP	AnalisesController: CRUD, transcrição e histórico. AnaliseController: rota anterior /analisar. DTOs de entrada e saída; GlobalExceptionHandler centraliza erros.
Aplicação	AnalisarTranscricaoUseCase: valida transcrição, chama Analisador e persiste. GerenciarAnalisesService: consulta, edição, exclusão e acompanhamento. ValidacaoAnalise: contrato e limites de armazenamento.
Domínio	AnaliseTranscricao: entidade imutável, UUID e regras comerciais. Pessoa, Cliente e Vendedor: participantes. Enums tipam decisões. Analisador e AnaliseRepository são portas utilizadas de fato.
Infraestrutura	GeminiAnalisador implementa Analisador; Jackson interpreta JSON. AnaliseDAO implementa AnaliseRepository usando JDBC e transações. ConexaoBanco fornece conexões Oracle.

Fluxos principais

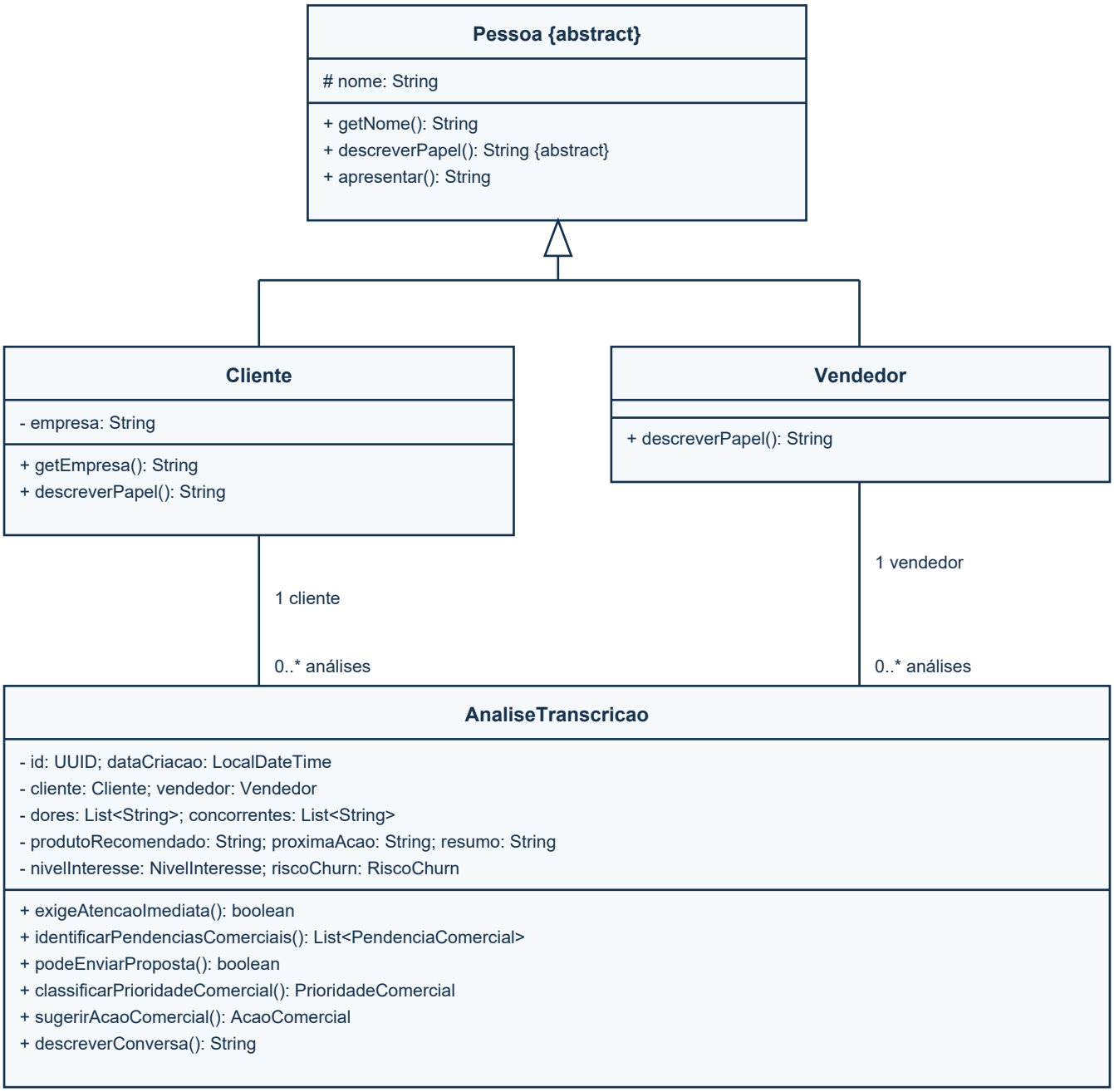
Criação: HTTP → controller → caso de uso → Analisador/Gemini → entidade → AnaliseRepository/DAO → Oracle → resposta 201.

Correção: HTTP → controller → service → validação → leitura da entidade → nova instância com o mesmo UUID e data → atualização transacional → resposta 200.

A aplicação depende das interfaces do domínio. Isso permite isolar Gemini nos testes e trocar o mecanismo de persistência sem introduzir JDBC ou HTTP nas regras de negócio. PreparedStatement parametriza os valores; recursos JDBC são fechados e falhas transacionais causam rollback.

Modelagem UML do domínio

Recorte estrutural das classes centrais. Convenções UML: + público, - privado, # protegido. Triângulo vazio aponta para a superclasse; associação possui multiplicidade.



Os atributos da análise são final e as listas são copiadas. Cliente e Vendedor sobrescrevem descreverPapel(); apresentar() usa despacho polimórfico. A entidade mantém identidade pelo UUID e não expõe setters.

Mapeamento das cinco tabelas Oracle

Tabela	Representação em Java e relacionamento
ANALISE	AnaliseTranscricao e os participantes Cliente/Vendedor. ID numérico interno; UUID_ANALISE único é o identificador público. Dados de cliente e vendedor são colunas do agregado.
ANALISE_DOR	List dores. PK (ANALISE_ID, POSICAO); FK para ANALISE. Relação: uma análise possui zero ou muitas dores.
ANALISE_CONCORRENTE	List concorrentes. PK (ANALISE_ID, POSICAO); FK para ANALISE. Relação: uma análise possui zero ou muitos concorrentes.
TRANSCRICAO	Texto original recuperado pelo repositório e por TranscricaoResponse. FK única ANALISE_ID; no banco, uma análise pode ter zero ou uma transcrição. O caso de uso sempre grava uma.
STATUS_ACOMPANHAMENTO	RegistroAcompanhamento: status, observacao e dataMudanca. FK ANALISE_ID. Uma análise possui zero ou muitos eventos.

Integridade e ciclo de vida

As cinco tabelas existentes foram mantidas. PK, FK, UNIQUE, NOT NULL e CHECKs já definidos no schema continuam aplicados. A exclusão da análise remove os dependentes por ON DELETE CASCADE.

O INSERT salva análise, dores, concorrentes e transcrição em uma transação. O PUT substitui os dados editáveis e as coleções, preservando UUID, data de criação, texto original e eventos. O histórico é acrescentado por INSERT.

DATA_CRIACAO é mapeada como LocalDateTime, sem fuso, com precisão de microssegundos. Datas de acompanhamento são geradas pelo Oracle. Não foi criada uma classe artificial para cada tabela: coleções e texto são valores pertencentes ao agregado.

Contrato REST e exemplos

Método e caminho	Sucesso / finalidade
POST /analises	201 + Location; cria com Gemini
GET /analises	200; lista análises
GET /analises/{id}	200; detalhes e indicadores
PUT /analises/{id}	200; corrige campos editáveis
DELETE /analises/{id}	204; exclusão em cascata
GET /analises/{id}/transcricao	200; analiseId e textoOriginal
GET /analises/{id}/acompanhamentos	200; histórico cronológico
POST /analises/{id}/acompanhamentos	204; acrescenta evento
POST /analisar	200; criação pela rota anterior

Criação: envie application/json com o campo transcricao. O corpo precisa conter ao menos 20 caracteres após trim. A resposta inclui UUID, dataCriacao, dados da análise e indicadores calculados.

Acompanhamento: {"status":"EM_ANDAMENTO", "observacao":"Proposta em preparação"}. Status: PENDENTE, EM_ANDAMENTO, CONCLUIDO ou DESCARTADO.

PUT: envie cliente, empresaCliente, vendedor, dores, produtoRecomendado, nivelInteresse, riscoChurn, concorrentes, proximaAcao e resumo. O exemplo completo está em docs/api-rest.md e na coleção Postman.

Validações e regras de negócio

Validações e respostas previsíveis

A camada de aplicação exige cliente e vendedor, enums válidos e listas não nulas. Listas vazias são válidas; elementos vazios não. Campos textuais de produto, resumo e próxima ação podem estar vazios, mas precisam ser enviados no PUT.

Nomes e empresa: até 500 bytes UTF-8. Observação: até 1000 bytes. Esses limites respeitam VARCHAR2 BYTE em Oracle AL32UTF8. Dados de participantes retornados pela IA são verificados antes da persistência.

HTTP	Situação
400 / 404	Entrada inválida / análise inexistente.
405 / 415	Método não permitido / tipo de conteúdo inválido.
409	Conflito de integridade dos dados.
502 / 500	Falha de análise externa / falha interna ou de persistência.

Comportamentos no domínio

exigeAtencaoImediata: risco alto ou interesse alto com concorrentes.
identificarPendenciasComerciais: interesse/risco desconhecidos, ausência de dores ou produto.
podeEnviarProposta: interesse alto, risco baixo, dor identificada e produto.
classificarPrioridadeComercial: aplica precedência entre urgência, interesse, risco e pendências.
sugerirAcaoComercial: prioriza retenção, diagnóstico, objeções e preparação de proposta.

As regras já existentes foram preservadas. São uma política comercial determinística proposta pelo projeto, não um modelo estatístico validado. O detalhamento e as precedências estão em docs/regras-comerciais.md.

Execução, testes e requisitos

Executar: JDK 21, Oracle provisionado e chave Gemini válida. Na cópia distribuída configure DB_URL, DB_USERNAME, DB_PASSWORD e GEMINI_API_KEY. Rode Main.java ou mvnw.cmd spring-boot:run. Importe a coleção Postman. Não é necessário executar MainTerminal junto.

Build: mvnw.cmd verify. Resultado em 07/10/2026: 69 testes executados sem falhas; um teste Oracle ignorado por padrão. O build gerou o JAR com sucesso. O aviso Mockito/Byte Buddy da JVM não impediu a validação.

Oracle real: o teste opt-in OracleIntegracaoTest passou separadamente, verificando inserção, consulta, atualização, transcrição, acompanhamento, exclusão e limpeza do registro temporário. Não foram executadas chamadas Gemini reais nesta validação.

Critério	Evidência implementada
Model — 10 pontos	Entidade imutável, participantes e valores alinhados ao agregado e às cinco tabelas.
DAO e Service — 20 pontos	CRUD transacional existente, serviço de gerenciamento, validações e teste real no Oracle.
REST — 30 pontos	Nove operações documentadas, contratos JSON, códigos HTTP e coleção Postman.
Boas práticas — 20 pontos	Camadas separadas, interfaces usadas, polimorfismo, erros centralizados e testes automatizados.

Feedback 3: ConexaoAcademica demonstra credenciais literais fictícias no perfil academico. A conta precisa ser provisionada para funcionar; o Oracle foi testado com ConexaoBanco padrão. A aceitação acadêmica depende do professor.

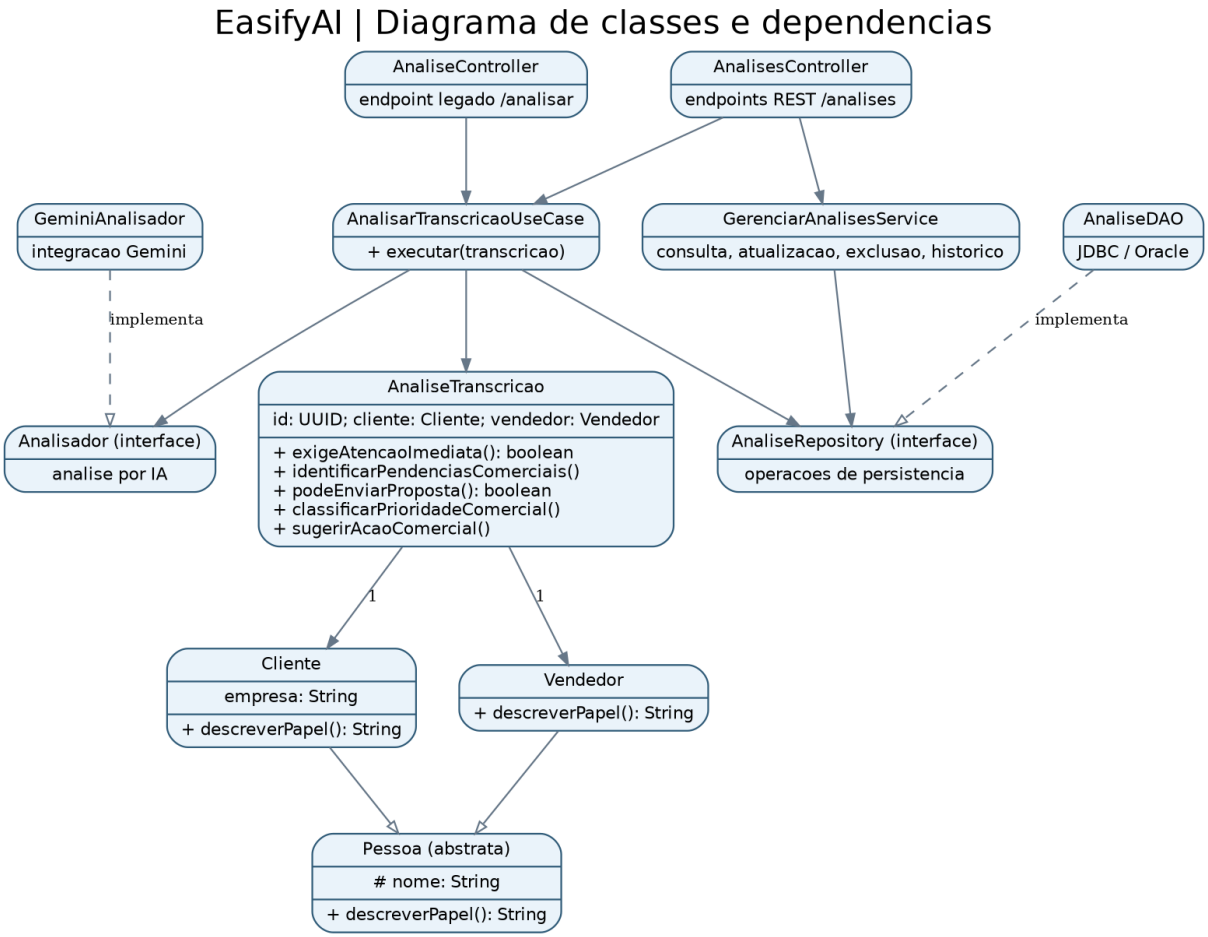
Entrega: código-fonte, guias, coleção e este PDF no ZIP. As credenciais pessoais não são incluídas. A conexão local original foi preservada. Sem front-end, autenticação ou paginação nesta versão.

EasifyAI — Atualizacao documental da Sprint 4

Complemento da documentacao original de Domain Driven Design — Java. Este adendo deve ser lido em conjunto com as paginas originais.

1. Diagrama de classes atualizado

O diagrama a seguir amplia o recorte de dominio original com controllers, casos de uso, servicos, interfaces e adaptadores de infraestrutura. Setas tracejadas indicam implementacoes de interfaces.



2. Contrato REST consolidado

Metodo	URI	Sucesso
POST	/analises	201
GET	/analises	200
GET	/analises/{id}	200
PUT	/analises/{id}	200

Metodo	URI	Sucesso
DELETE	/analises/{id}	204
GET	/analises/{id}/transcricao	200
GET	/analises/{id}/acompanhamentos	200
POST	/analises/{id}/acompanhamentos	204
POST	/analisar	200

Erros: a API possui tratamento centralizado de excecoes; o status exato de cada cenario deve ser confirmado pelos testes e pelo GlobalExceptionHandler.

3. Evidencias de interface e limites do escopo

A entrega inclui backend e colecao Postman. Nao ha telas de front-end no pacote analisado; capturas do Postman podem demonstrar as rotas, mas nao devem ser apresentadas como telas de uma interface implementada. O requisito de prints das telas implementadas permanece sujeito a avaliacao do professor.

4. Referencias documentais

O PDF original menciona docs/api-rest.md, arquivo que nao consta no pacote analisado. Para o contrato de API, consulte a tabela acima e docs/EasifyAI.postman_collection.json. Outros links do README para documentos ausentes devem ser corrigidos antes da entrega.

5. Validacao

Conforme informado pela equipe, os testes e a integracao Oracle funcionam. Este adendo nao representa uma nova execucao automatizada dos testes.